SQL Interview Questions by Table Schema

This handbook groups SQL interview questions based on related table schemas. Practice section by section to build stronger SQL intuition.

1. Employee/Department Questions

Table Schema

```
Employee(
    employee_id INT,
    name VARCHAR(100),
    salary INT,
    dept_id INT,
    manager_id INT,
    hire_date DATE
)

Department(
    dept_id INT,
    dept_name VARCHAR(100)
)
```

Interview Questions

Q1. Find employees whose salary is above company average

SQL: SELECT * FROM Employee WHERE salary > (SELECT AVG(salary) FROM Employee);

Pattern: Subquery + Aggregate

Q2. Find employees who joined in the last 30 days

SQL: SELECT * FROM Employee WHERE hire_date >= CURRENT_DATE - INTERVAL 30 DAY;

Pattern: Date Function + Filtering

Q3. Find employees who have the same salary

SQL: SELECT salary, COUNT(*) AS cnt FROM Employee GROUP BY salary HAVING COUNT(*) > 1;

Pattern: GROUP BY + HAVING

Q4. Display employees who earn more than their manager

SQL: SELECT e.* FROM Employee e JOIN Employee m ON e.manager_id = m.employee_id WHERE e.salary > m.salary;

Pattern: Self Join + Filtering

Q5. Split full name into first and last name

SQL: SELECT SUBSTRING_INDEX(full_name, ' ', 1) AS first_name, SUBSTRING_INDEX(full_name, ' ', -1) AS last_name FROM Employee;

Pattern: String Function

Q6. Calculate median salary in each department

SQL: SELECT dept_id, AVG(salary) AS median_salary FROM (SELECT dept_id, salary, ROW_NUMBER() OVER (PARTITION BY dept_id ORDER BY salary) AS rn, COUNT(*) OVER (PARTITION BY dept_id) AS cnt FROM Employee) t WHERE rn IN (FLOOR((cnt+1)/2), CEIL((cnt+1)/2)) GROUP BY dept_id;

Pattern: Window Function + Subquery

Q7. Display employees who report to managers in different departments

SQL: SELECT e.* FROM Employee e JOIN Employee m ON e.manager_id = m.employee_id WHERE e.dept_id <> m.dept_id;

Pattern: Self Join + Filtering

Q8. Display employees hired on the same date

SQL: SELECT hire_date, COUNT(*) AS emp_count FROM Employee GROUP BY hire_date HAVING COUNT(*) > 1;

Pattern: GROUP BY + HAVING

Q9. Find Nth highest salary without using LIMIT

SQL: SELECT DISTINCT salary FROM Employee e1 WHERE (SELECT COUNT(DISTINCT salary) FROM Employee e2 WHERE e2.salary > e1.salary) = N-1;

Pattern: Correlated Subquery

Q10. Find the second highest salary from the Employee table

SQL: SELECT MAX(salary) AS second_highest FROM Employee WHERE salary < (SELECT MAX(salary) FROM Employee);

Pattern: Subquery + Aggregate

Q11. Display top 3 highest paid employees in each department

SQL: SELECT * FROM (SELECT e.*, DENSE_RANK() OVER (PARTITION BY dept_id ORDER BY salary DESC) AS rnk FROM Employee e) t WHERE rnk <= 3;

Pattern: Window Function + PARTITION BY

Q12. Find gaps in sequence of IDs

SQL: SELECT e1.id + 1 AS gap_start FROM Employee e1 WHERE NOT EXISTS (SELECT 1 FROM Employee e2 WHERE e2.id = e1.id + 1) AND e1.id < (SELECT MAX(id) FROM Employee);

Pattern: Subquery + NOT EXISTS

Q13. Find names that contain numbers

SQL: SELECT * FROM Employee WHERE name REGEXP '[0-9]';

Pattern: String Pattern + Filtering

Q14. Calculate salary difference between employee and department average

SQL: SELECT e.*, e.salary - AVG(e.salary) OVER (PARTITION BY dept_id) AS diff FROM Employee e;

Pattern: Window Function + Aggregate

Q15. Find the highest salary in each department

SQL: SELECT dept_id, MAX(salary) AS highest_salary FROM Employee GROUP BY dept_id;

Pattern: GROUP BY + Aggregate

Q16. Find employees hired in same month and year

SQL: SELECT YEAR(hire_date) AS year, MONTH(hire_date) AS month, COUNT(*) AS count FROM Employee GROUP BY YEAR(hire_date), MONTH(hire_date) HAVING COUNT(*) > 1;

Pattern: Date Function + GROUP BY + HAVING

Q17. Display employee hierarchy using recursive CTE

SQL: WITH RECURSIVE emp_hierarchy AS (SELECT employee_id, name, manager_id, 1 AS level FROM Employee WHERE manager_id IS NULL UNION ALL SELECT e.employee_id, e.name, e.manager_id, eh.level+1 FROM Employee e JOIN emp_hierarchy eh ON e.manager_id = eh.employee_id) SELECT * FROM emp_hierarchy;

Pattern: Recursive CTE + JOIN

Q18. Find employees with anniversaries in current month

SQL: SELECT * FROM Employee WHERE MONTH(hire_date) = MONTH(CURRENT_DATE);

Pattern: Date Function + Filtering

Q19. Find overlapping employee work shifts

SQL: SELECT s1.employee_id, s2.employee_id FROM Shift s1 JOIN Shift s2 ON s1.shift_id <> s2.shift_id AND s1.start_time < s2.end_time AND s1.end_time > s2.start_time;

Pattern: Self Join + Time Logic

Q20. Delete duplicate rows keeping only one record

SQL: DELETE FROM Employee WHERE id NOT IN (SELECT min_id FROM (SELECT MIN(id) AS min_id FROM Employee GROUP BY name, salary, department) AS t);

Pattern: Subquery + DELETE

Q21. Find departments with no employees

SQL: SELECT d.* FROM Department d LEFT JOIN Employee e ON d.dept_id = e.dept_id WHERE e.employee_id IS NULL;

Pattern: LEFT JOIN + NULL check

Q22. Find departments where all employees earn above 50k

SQL: SELECT dept_id FROM Employee GROUP BY dept_id HAVING MIN(salary) > 50000;

Pattern: GROUP BY + HAVING + Aggregate

2. Customer/Orders Questions

Table Schema

```
Customer(
    customer_id INT,
    name VARCHAR(100),
    email VARCHAR(100),
    phone VARCHAR(20)
)

Orders(
    order_id INT,
    customer_id INT,
    order_amount DECIMAL(10,2),
    order_date DATE
)
```

Interview Questions

Q1. Find longest streak of consecutive days with orders

SQL: WITH dates AS (SELECT DISTINCT order_date FROM Orders), numbered AS (SELECT order_date, ROW_NUMBER() OVER (ORDER BY order_date) - ROW_NUMBER() OVER (PARTITION BY 1 ORDER BY order_date) AS grp FROM dates) SELECT MAX(cnt) FROM (SELECT COUNT(*) AS cnt FROM numbered GROUP BY grp) t;

Pattern: Window Function + Subquery

Q2. Calculate average time between orders per customer

SQL: SELECT customer_id, AVG(DATEDIFF(order_date, prev_order)) AS avg_days FROM (SELECT customer_id, order_date, LAG(order_date) OVER (PARTITION BY customer_id ORDER BY order_date) AS prev_order FROM Orders) t WHERE prev_order IS NOT NULL GROUP BY customer_id;

Pattern: Window Function + Date + Subquery

Q3. Display customers with more than 5 orders

SQL: SELECT customer_id, COUNT(*) AS order_count FROM Orders GROUP BY customer_id HAVING COUNT(*) > 5;

Pattern: GROUP BY + HAVING

Q4. Calculate churn rate by cohort

SQL: SELECT cohort_month, COUNT(CASE WHEN last_order_month < cohort_month + INTERVAL 3 MONTH THEN 1 END)*100.0/COUNT(*) AS churn_rate FROM CustomerCohort GROUP BY cohort_month;

Pattern: CASE + Date + Aggregate

Q5. Find the customer with highest total purchase amount

SQL: SELECT customer_id, SUM(order_amount) AS total FROM Orders GROUP BY customer_id ORDER BY total DESC LIMIT 1;

Pattern: GROUP BY + Aggregate + ORDER BY

Q6. Display total order amount for each customer

SQL: SELECT customer_id, SUM(order_amount) AS total FROM Orders GROUP BY customer_id;

Pattern: GROUP BY + Aggregate

Q7. Calculate customer retention rate month-over-month

SQL: SELECT m.month, COUNT(DISTINCT m_next.customer_id)*100.0/COUNT(DISTINCT m.customer_id) AS retention FROM (SELECT DATE_TRUNC('month', order_date) AS month, customer_id FROM Orders) m LEFT JOIN (SELECT DATE_TRUNC('month', order_date) AS month, customer_id FROM Orders) m_next ON m.customer_id = m_next.customer_id AND m_next.month = m.month + INTERVAL '1 month' GROUP BY m.month;

Pattern: Self Join + Date + Aggregate

Q8. Extract domain from email addresses

SQL: SELECT email, SUBSTRING(email, POSITION('@' IN email) + 1) AS domain FROM Customer;

Pattern: String Function

Q9. Calculate business days between two dates

SQL: SELECT order_id, DATEDIFF(delivery_date, order_date) - (WEEK(delivery_date) - WEEK(order_date))*2 AS business_days FROM Orders;

Pattern: Date Function + Calculation

Q10. Find all orders placed on weekends

SQL: SELECT * FROM Orders WHERE DAYOFWEEK(order_date) IN (1,7);

Pattern: Date Function + Filtering

Q11. Find phone numbers not matching pattern

SQL: SELECT * FROM Customer WHERE phone NOT REGEXP '^[0-9]{3}-[0-9]{3}-[0-9]{4}$';

Pattern: String Pattern + Filtering

Q12. Find all emails with invalid format

SQL: SELECT * FROM Customer WHERE email NOT LIKE '%@%.%' OR email LIKE '%@%@%';

Pattern: String Pattern + Filtering

Q13. Display orders grouped by fiscal quarter

SQL: SELECT QUARTER(order_date) AS quarter, YEAR(order_date) AS year, COUNT(*) FROM Orders GROUP BY YEAR(order_date), QUARTER(order_date);

Pattern: Date Function + GROUP BY

Q14. Find customers who placed orders in consecutive months

SQL: SELECT DISTINCT o1.customer_id FROM Orders o1 JOIN Orders o2 ON o1.customer_id = o2.customer_id AND MONTH(o2.order_date) = MONTH(o1.order_date) + 1 AND YEAR(o2.order_date) = YEAR(o1.order_date);

Pattern: Self Join + Date Function

Q15. Find average order value per customer

SQL: SELECT customer_id, AVG(order_amount) AS avg_order FROM Orders GROUP BY customer_id;

Pattern: GROUP BY + Aggregate

Q16. Find customers who ordered the same product more than once

SQL: SELECT o.customer_id, od.product_id, COUNT(*) AS times FROM Orders o JOIN OrderDetails od ON o.order_id = od.order_id GROUP BY o.customer_id, od.product_id HAVING COUNT(*) > 1;

Pattern: JOIN + GROUP BY + HAVING

Q17. Find first and last order date for each customer

SQL: SELECT customer_id, MIN(order_date) AS first_order, MAX(order_date) AS last_order FROM Orders GROUP BY customer_id;

Pattern: GROUP BY + Aggregate

Q18. Find duplicate records across multiple columns

SQL: SELECT name, email, COUNT(*) FROM Customer GROUP BY name, email HAVING COUNT(*) > 1;

Pattern: GROUP BY + HAVING

Q19. Find customers who purchased in Q1 but not Q2

SQL: SELECT DISTINCT customer_id FROM Orders WHERE QUARTER(order_date)=1 AND customer_id NOT IN (SELECT customer_id FROM Orders WHERE QUARTER(order_date)=2);

Pattern: Subquery + NOT IN + Date

Q20. Find customers with increasing order values

SQL: SELECT customer_id FROM (SELECT customer_id, order_date, order_amount, LAG(order_amount) OVER (PARTITION BY customer_id ORDER BY order_date) AS prev_amount FROM Orders) t WHERE order_amount > prev_amount GROUP BY customer_id HAVING COUNT(*) = (SELECT COUNT(*) - 1 FROM Orders o WHERE o.customer_id = customer_id);

Pattern: Window Function + Subquery + HAVING

Q21. List orders placed in the last quarter

SQL: SELECT * FROM Orders WHERE order_date >= DATE_SUB(DATE_SUB(CURRENT_DATE, INTERVAL (MONTH(CURRENT_DATE)-1) MONTH), INTERVAL 3 MONTH);

Pattern: Date Function + Filtering

Q22. Find customers who have never placed an order

SQL: SELECT c.* FROM Customer c LEFT JOIN Orders o ON c.customer_id = o.customer_id WHERE o.order_id IS NULL;

Pattern: LEFT JOIN + NULL check

Q23. Find customers who ordered all available products

SQL: SELECT customer_id FROM Orders o JOIN OrderDetails od ON o.order_id = od.order_id GROUP BY customer_id HAVING COUNT(DISTINCT od.product_id) = (SELECT COUNT(*) FROM Product);

Pattern: JOIN + GROUP BY + HAVING

Q24. Display month-over-month growth in orders

SQL: SELECT month, orders, orders - COALESCE(prev_orders,0) AS growth FROM (SELECT MONTH(order_date) AS month, COUNT(*) AS orders FROM Orders GROUP BY MONTH(order_date)) t LEFT JOIN (SELECT MONTH(order_date) AS month, COUNT(*) AS prev_orders FROM Orders GROUP BY MONTH(order_date)) p ON t.month = p.month + 1;

Pattern: Window Function + Date + Aggregate

3. Product/Sales Questions

Table Schema

```
Product(
    product_id INT,
    category VARCHAR(100),
    price DECIMAL(10,2),
    stock_quantity INT
)

OrderDetails(
    order_id INT,
    product_id INT,
    quantity INT,
    price DECIMAL(10,2)
)

Sales(
    product_id INT,
    region VARCHAR(50),
    sales DECIMAL(10,2),
    order_date DATE
)
```

Interview Questions

Q1. Display products never sold in a specific region

SQL: SELECT p.* FROM Product p WHERE p.product_id NOT IN (SELECT DISTINCT product_id FROM Sales WHERE region = 'North');

Pattern: Subquery + NOT IN + Filtering

Q2. Find products with price changes

SQL: SELECT p1.product_id, p1.price AS old_price, p2.price AS new_price FROM ProductHistory p1 JOIN ProductHistory p2 ON p1.product_id = p2.product_id AND p1.effective_date < p2.effective_date WHERE p1.price <> p2.price;

Pattern: Self Join + Filtering

Q3. Display products with declining inventory

SQL: SELECT product_id FROM Inventory i WHERE i.stock_date = CURRENT_DATE AND i.quantity < (SELECT quantity FROM Inventory i2 WHERE i2.product_id = i.product_id AND i2.stock_date = CURRENT_DATE - INTERVAL 1 MONTH);

Pattern: Correlated Subquery + Date

Q4. Find products with sales greater than average

SQL: SELECT product_id, SUM(quantity * price) AS sales FROM OrderDetails GROUP BY product_id HAVING SUM(quantity * price) > (SELECT AVG(total) FROM (SELECT SUM(quantity * price) AS total FROM OrderDetails GROUP BY product_id) AS t);

Pattern: Subquery + GROUP BY + HAVING

Q5. Find products that are out of stock

SQL: SELECT * FROM Product WHERE stock_quantity = 0 OR stock_quantity IS NULL;

Pattern: Filtering + NULL check

Q6. Display products never returned

SQL: SELECT p.* FROM Product p WHERE p.product_id NOT IN (SELECT product_id FROM Returns);

Pattern: Subquery + NOT IN

Q7. Display running difference from previous month

SQL: SELECT month, sales, sales - LAG(sales) OVER (ORDER BY month) AS diff FROM MonthlySales;

Pattern: Window Function + Calculation

Q8. Calculate running total of sales by date

SQL: SELECT order_date, order_amount, SUM(order_amount) OVER (ORDER BY order_date ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW) AS running_total FROM Orders;

Pattern: Window Function + Aggregate

Q9. Find products ordered together frequently

SQL: SELECT od1.product_id AS product1, od2.product_id AS product2, COUNT(*) AS frequency FROM OrderDetails od1 JOIN OrderDetails od2 ON od1.order_id = od2.order_id AND od1.product_id < od2.product_id GROUP BY od1.product_id, od2.product_id ORDER BY frequency DESC;

Pattern: Self Join + GROUP BY

Q10. Display year-to-date sales by month

SQL: SELECT MONTH(order_date) AS month, SUM(order_amount) AS monthly_sales, SUM(SUM(order_amount)) OVER (ORDER BY MONTH(order_date)) AS ytd_sales FROM Orders WHERE YEAR(order_date)=YEAR(CURRENT_DATE) GROUP BY MONTH(order_date);

Pattern: Window Function + Date + Aggregate

Q11. Calculate percentage of total sales by region

SQL: SELECT region, SUM(sales)*100.0/(SELECT SUM(sales) FROM Sales) AS percentage FROM Sales GROUP BY region;

Pattern: Subquery + Aggregate + Calculation

Q12. List products in at least 10 different orders

SQL: SELECT product_id, COUNT(DISTINCT order_id) AS order_count FROM OrderDetails GROUP BY product_id HAVING COUNT(DISTINCT order_id) >= 10;

Pattern: GROUP BY + HAVING + DISTINCT

Q13. Display year-over-year sales comparison

SQL: SELECT YEAR(order_date) AS year, SUM(order_amount) AS sales, LAG(SUM(order_amount)) OVER (ORDER BY YEAR(order_date)) AS prev_year FROM Orders GROUP BY YEAR(order_date);

Pattern: Window Function + Date + Aggregate

Q14. Calculate cumulative sum partitioned by category

SQL: SELECT category, order_date, sales, SUM(sales) OVER (PARTITION BY category ORDER BY order_date) AS cumulative FROM Sales;

Pattern: Window Function + PARTITION BY

Q15. Find product category with highest revenue

SQL: SELECT p.category, SUM(od.quantity * od.price) AS revenue FROM Product p JOIN OrderDetails od ON p.product_id = od.product_id GROUP BY p.category ORDER BY revenue DESC LIMIT 1;

Pattern: JOIN + GROUP BY + Aggregate

Q16. Find products with declining sales trend

SQL: SELECT product_id FROM (SELECT product_id, MONTH(order_date) AS month, SUM(quantity) AS sales, LAG(SUM(quantity)) OVER (PARTITION BY product_id ORDER BY MONTH(order_date)) AS prev_sales FROM OrderDetails GROUP BY product_id, MONTH(order_date)) t WHERE sales < prev_sales;

Pattern: Window Function + Subquery + Filtering

Q17. Find products sold in all regions

SQL: SELECT product_id FROM Sales GROUP BY product_id HAVING COUNT(DISTINCT region) = (SELECT COUNT(DISTINCT region) FROM Sales);

Pattern: GROUP BY + HAVING + Subquery

Q18. Display top 5 best-selling products

SQL: SELECT product_id, SUM(quantity) AS total_sold FROM OrderDetails GROUP BY product_id ORDER BY total_sold DESC LIMIT 5;

Pattern: GROUP BY + Aggregate + ORDER BY

Q19. Display pivot table of sales by product and month

SQL: SELECT product_id, SUM(CASE WHEN MONTH(order_date)=1 THEN quantity ELSE 0 END) AS Jan, SUM(CASE WHEN MONTH(order_date)=2 THEN quantity ELSE 0 END) AS Feb FROM OrderDetails od JOIN Orders o ON od.order_id = o.order_id GROUP BY product_id;

Pattern: CASE + GROUP BY + Aggregate

Q20. List products that have never been ordered

SQL: SELECT p.* FROM Product p LEFT JOIN OrderDetails od ON p.product_id = od.product_id WHERE od.order_id IS NULL;

Pattern: LEFT JOIN + NULL check

Q21. Display top 3 products by revenue in each category

SQL: SELECT * FROM (SELECT category, product_id, revenue, ROW_NUMBER() OVER (PARTITION BY category ORDER BY revenue DESC) AS rn FROM ProductRevenue) t WHERE rn <= 3;

Pattern: Window Function + PARTITION BY

4. User/Activity Questions

Table Schema

```
UserLogin(
    user_id INT,
    login_date DATE
)

UserActivity(
    user_id INT,
    source VARCHAR(50),
    purchased INT
```

```
)
```

Interview Questions

Q1. Find users who logged in on consecutive days

SQL: SELECT DISTINCT u1.user_id FROM UserLogin u1 JOIN UserLogin u2 ON u1.user_id = u2.user_id AND u2.login_date = u1.login_date + INTERVAL 1 DAY;

Pattern: Self Join + Date Logic

Q2. Calculate conversion rate by traffic source

SQL: SELECT source, COUNT(CASE WHEN purchased=1 THEN 1 END)*100.0/COUNT(*) AS conversion_rate FROM UserActivity GROUP BY source;

Pattern: CASE + GROUP BY + Aggregate

5. Student/Academic Questions

Table Schema

```
Student(
    student_id INT,
    name VARCHAR(100),
    class_id INT,
    score INT
)
```

Interview Questions

Q1. List all students who scored above class average

SQL: SELECT s.student_id, s.name, s.score FROM Student s WHERE s.score > (SELECT AVG(score) FROM Student WHERE class_id = s.class_id);

Pattern: Correlated Subquery + Filtering

6. Transaction Questions

Table Schema

```
Transactions(
    transaction_id INT,
    account_id INT,
    amount DECIMAL(10,2),
    transaction_date DATE,
    transaction_time TIME
)
```

Interview Questions

Q1. Find busiest hour of day for transactions

SQL: SELECT HOUR(transaction_time) AS hour, COUNT(*) AS transaction_count FROM Transactions GROUP BY HOUR(transaction_time) ORDER BY transaction_count DESC LIMIT 1;

Pattern: Date Function + GROUP BY

Q2. Find transactions that exceed daily limit

SQL: SELECT account_id, transaction_date, SUM(amount) AS daily_total FROM Transactions GROUP BY account_id, transaction_date HAVING SUM(amount) > 10000;

Pattern: GROUP BY + HAVING + Aggregate

7. Booking/Reservation Questions

Table Schema

```
Booking(
    booking_id INT,
    room_id INT,
    start_date DATE,
    end_date DATE
)
```

Interview Questions

Q1. Find overlapping date ranges in bookings

SQL: SELECT b1.* FROM Booking b1 JOIN Booking b2 ON b1.room_id = b2.room_id AND b1.booking_id <> b2.booking_id AND b1.start_date <= b2.end_date AND b1.end_date >= b2.start_date;

Pattern: Self Join + Date Logic